The AKS primality test

Terence Tao · 11 August 2009

https://terrytao.wordpress.com/2009/08/11/the-aks-primality-test/

tao2typ · converted to Typst 2026-10-14 (UTC)

The Agrawal-Kayal-Saxena (AKS) primality test, discovered in 2002, is the first provably deterministic algorithm to determine the primality of a given number with a run time which is guaranteed to be polynomial in the number of digits, thus, given a large number n, the algorithm will correctly determine whether that number is prime or not in time $O(\log^{O(1)} n)$. (Many previous primality testing algorithms existed, but they were either probabilistic in nature, had a running time slower than polynomial, or the correctness could not be guaranteed without additional hypotheses such as GRH.)

The AKS test is of some relevance to the polymath project "Finding primes", so I thought I would sketch the details of the test (and the proof that it works) here. (Of course, full details can be found in the original paper, which is nine pages in length and almost entirely elementary in nature.) It relies on polynomial identities that are true modulo n when n is prime, but cannot hold for n non-prime as they would generate a large number of additional polynomial identities, eventually violating the factor theorem (which asserts that a polynomial identity of degree at most d can be obeyed by at most d values of the unknown).

Our starting point is Fermat's little theorem, which asserts that

$$a^p = a \bmod p \tag{1}$$

for every prime p and every a. This theorem suggests an obvious primality test: to test whether a number n is prime, pick a few values of a and see whether $a^n = a \bmod n$. (Note that a^n can be computed in time $O(\log^{O(1)} n)$ for any fixed a by expressing n in binary, and repeatedly squaring a.) If the statement $a^n = a \bmod n$ fails for some a, then n would be composite. Unfortunately, the converse is not true: there exist non-prime numbers n, known as Carmichael numbers, for which $a^n = a \bmod n$ for all a coprime to n (561 is the first example). So Fermat's little theorem cannot be used, by itself, to establish primality for general n, because it is too weak to eliminate all non-prime numbers. (The situation improves though for more special types of n, such as Mersenne numbers; see my earlier post on the Lucas-Lehmer test for more discussion.)

However, there is a stronger version of Fermat's little theorem which does eliminate all non-prime numbers. Specifically, if p is prime and a is arbitrary, then one has the polynomial identity

$$(X + a)^p = X^p + a \bmod p \tag{2}$$

where X is an indeterminate variable. (More formally, we have the identity $(X + a)^p = X^p + a$ in the ring $F_p[X]$ of polynomials of one variable X over the finite field F_p of p elements.) This identity (a manifestation of the Frobenius endomorphism) clearly implies (1) by setting $X = 0$; conversely, one can easily deduce (2) from (1) by expanding out $(X + a)^p$ using the binomial

theorem and the observation that the binomial coefficients $\binom{p}{i} = \frac{p \cdot \ldots \cdot (p-i+1)}{i!}$ are divisible by p for all $1 \leq i < p$. Conversely, if

$$(X+a)^n = X^n + a \bmod n \tag{3}$$

(i.e. $(X+a)^n = X^n + a$ in $(\mathbb{Z}/n\mathbb{Z})[X]$) for some a coprime to n, then by comparing coefficients using the binomial theorem we see that $\binom{n}{i}$ is divisible by n for all $1 \leq i < n$. But if n is divisible by some smaller prime p, then by setting i equal to the largest power of p that divides n, one sees that $\binom{n}{i}$ is not divisible by enough powers of p to be divisible by n, a contradiction. Thus one can use (3) (for a single value of a coprime to n) to decide whether n is prime or not.

Unfortunately, this algorithm, while deterministic, is not polynomial-time, because the polynomial $(X+a)^n$ has $n+1$ coefficients and will therefore take at least $O(n)$ time to compute. However, one can speed up the process by descending to a quotient ring of $(\mathbb{Z}/n\mathbb{Z})[X]$, such as $\mathbb{Z}/n\mathbb{Z}[X]/(X^r - 1)$ for some r. Clearly, if the identity $(X+a)^n = X^n + a$ holds in $(\mathbb{Z}/n\mathbb{Z})[X]$, then it will also hold in $(\mathbb{Z}/n\mathbb{Z})[X]/(X^r - 1)$, thus

$$(X+a)^n = X^n + a \bmod n, X^r - 1. \tag{4}$$

The point of doing this is that (if r is not too large) the left-hand side of (4) can now be computed quickly (again by expanding n in binary and performing repeated squaring), because all polynomials can be reduced to be of degree less than r, rather than being as large as n. Indeed, if $r = O(\log^{O(1)} n)$, then one can test (4) in time $O(\log^{O(1)} n)$.

We are not done yet, because it could happen that (4) holds but (3) fails. But we have the following key theorem:

> **Theorem 1 (AKS theorem)** Suppose that for all $1 \leq a, r \leq O(\log^{O(1)} n)$, (4) holds, and a is coprime to n. Then n is either a prime, or a power of a prime.

Of course, coprimality of a and n can be quickly tested using the Euclidean algorithm, and if coprimality fails then n is of course composite. Also, it is easy to quickly test for the property that n is a power of an integer (just compute the roots $n^{1/k}$ for $1 \leq k \leq \log_2 n$), and such powers are clearly composite. From all this (and (2), one soon sees that theorem gives rise to a deterministic polynomial-time test for primality. One can optimise the powers of $\log n$ in the bounds for a, r (as is done in the original paper), but we will not do so here to keep the exposition uncluttered.

Actually, we don't need (4) satisfied for all that many exponents r to make the theorem work; just one well-chosen r will do. More precisely, we have

> **Theorem 2 (AKS theorem, key step)** Let r be coprime to n, and such that n has order greater than $\log_2^2 n$ in the multiplicative group $(\mathbb{Z}/r\mathbb{Z})^\times$ (i.e. the residues $n^i \bmod r$ for $1 \leq i \leq \log^2 n$ are distinct). Suppose that for all $1 \leq a \leq O(r \log^{O(1)} n)$, (4) holds, and a is coprime to n. Then n is either a prime, or a power of a prime.

To find an r with the above properties we have

Lemma 3 (Existence of good r) There exists $r = O(\log^{O(1)} n)$ coprime to n, such that n has order greater than $\log_2^2 n$ in $(\mathbb{Z}/r\mathbb{Z})^\times$.

Proof: For each $1 \leq i \leq \log_2^2 n$, the number $n^i - 1$ has at most $O(\log^{O(1)} n)$ prime divisors (by the fundamental theorem of arithmetic). If one picks r to be the first prime not equal to any of these prime divisors, one obtains the claim. (One can use a crude version of the prime number theorem to get the upper bound on r.) □

It is clear that Theorem 1 follows from Theorem 2 and Lemma 3, so it suffices now to prove Theorem 2.

Suppose for contradiction that Theorem 2 fails. Then n is divisible by some smaller prime p, but is not a power of p. Since n is coprime to all numbers of size $O(\log^{O(1)} n)$ we know that p is not of polylogarithmic size, thus we may assume $p \geq \log^C n$ for any fixed C. As r is coprime to n, we see that r is not a multiple of p (indeed, one should view p as being much larger than r).

Let F be a field extension of F_p by a primitive r^{th} root of unity X, thus $F = F_p[X]/h(X)$ for some factor $h(X)$ (in $F_p[X]$) of the rth cyclotomic polynomial $\Phi_r(X)$. From the hypothesis (4), we see that

$$(X + a)^n = X^n + a$$

in F for all $1 \leq a \leq A$, where $A = O(r \log^{O(1)} n)$. Note that n is coprime to every integer less than A, and thus $A < p$.

Meanwhile, from (2) one has

$$(X + a)^p = X^p + a$$

in F for all such a. The two equations give

$$(X^p + a)^{n/p} = (X^p)^{n/p} + a.$$

Note that the pth power X^p of a primitive r^{th} root of unity X is again a primitive r^{th} root of unity (and conversely, every primitive r^{th} root arises in this fashion) and hence we also have

$$(X + a)^{n/p} = X^{n/p} + a$$

in F for all $1 \leq a \leq A$.

Inspired by this, we define a key concept: a positive integer m coprime to r is said to be *introspective* if one has

$$(X + a)^m = X^m + a$$

in F for all $1 \leq a \leq A$, or equivalently if $(X + a)^m = \phi_m(X + a)$, where $\phi_m : F \to F$ is the ring homomorphism that sends X to X^m.

We have just shown that $p, n, n/p$ are all introspective; 1 is also trivially introspective. Furthermore, if m and m' are introspective, it is not hard to see that mm' is also introspective. Thus we in fact have a lot of introspective integers: any number of the form $p^i(n/p)^j$ for $i, j \geq 0$ is introspective.

It turns out in fact that it is not possible to create so many different introspective numbers, basically the presence of so many polynomial identities in the field would eventually violate the factor theorem. To see this, let $\mathcal{G} \subset F^\times$ be the multiplicative group generated by the quantities $X+a$ for $1 \leq a \leq A$. Observe that $z^m = \phi_m(z)$ for all $z \in \mathcal{G}$. We now show that this places incompatible lower and upper bounds on $\mathcal{G}$. We begin with the lower bound:

> **Proposition 4 (Lower bound on $\mathcal{G}$)** $|\mathcal{G}| \geq 2^t$.

Proof: Let $P(X)$ be a product of less than t of the quantities $X+1, \ldots, X+A$ (allowing repetitions), then $P(X)$ lies in $\mathcal{G}$. Since $A \geq 2r \geq 2t$, there are certainly at least 2^t ways to pick such a product. So to establish the proposition it suffices to show that all these products are distinct.

Suppose for contradiction that $P(X) = Q(X)$, where P, Q are different products of less than t of the $X+1, \ldots, X+A$. Then, for every introspective m, $P(X^m) = Q(X^m)$ as well (note that $P(X^m) = \phi_m(P(X))$). In particular, this shows that $X^{m_1}, \ldots, X^{m_t}$ are all roots of the polynomial $P-Q$. But this polynomial has degree less than t, and the $X^{m_1}, \ldots, X^{m_t}$ are distinct by hypothesis, and we obtain the desired contradiction by the factor theorem. □

> **Proposition 5 (Upper bound on $\mathcal{G}$)** Suppose that there are exactly t residue classes modulo r of the form $p^i(n/p)^j \mod r$ for $i, j \geq 0$. Then $|\mathcal{G}| \leq n^{\sqrt{t}}$.

Proof: By the pigeonhole principle, we must have a collision

$$p^i(n/p)^j = p^{i'}(n/p)^{j'} \mod r$$

for some $0 \leq i, j, i', j' \leq \sqrt{t}$ with $(i,j) \neq (i',j')$. Setting $m := p^i(n/p)^j$ and $m' := p^{i'}(n/p)^{j'}$, we thus see that there are two distinct introspective numbers m, m' of size most $n^{\sqrt{t}}$ which are equal modulo r. (To ensure that m, m' are distinct, we use the hypothesis that n is not a power of p.) This implies that $\phi_m = \phi_{m'}$, and thus $z^m = z^{m'}$ for all $z \in \mathcal{G}$. But the polynomial $z^m - z^{m'}$ has degree at most $n^{\sqrt{t}}$, and the claim now follows from the factor theorem. □

Since n has order greater than $\log^2 n$ in $(\mathbb{Z}/r\mathbb{Z})^\times$, we see that the number t of residue classes r of the form $p^i(n/p)^j$ is at least $\log^2 n$. But then $2^t > n^{\sqrt{t}}$, and so Propositions 4, 5 are incompatible.

Update, Aug 12: A thorough discussion of the AKS algorithm by Andrew Granville for Bull. Amer. Math. Soc. can be found here.

Comments (31)



Henry · 11 August 2009

I like that this proof illustrates how fast (i.e. polynomial) computability fails to be analogous to general computability. If one has an "elementary" proof that a witness with some computably checkable property exists, the proof can generally be turned into a computation of that witness. (This can be made formal: if the proof can be formulated, even in a theory like full second order arithmetic, it can be converted to a computation of the witness.)

Yet, when given a composite number, the AKS primality test gives a fairly simple demonstration, in polynomial time, that there is some factor, but no way to quickly find what that factor is. Interestingly, the test does give a witness to *something*, and it's the task of converting that witness into a factor which cannot be carried out quickly (in polynomial time).

 **windfarmmusic** · 11 August 2009

In the parenthetical at the end of the first paragraph, it seems like 'faster' should read 'slower'. *[Fixed, thanks – T.]*

>  **windfarmmusic** · 11 August 2009
>
> Never mind!

 **theoreticalminimum** · 11 August 2009

Note: The url to the AKS paper has a backward slash after "primality" which has to be removed to get the correct url. *[Fixed, thanks – T.]*

 cipher3d · 11 August 2009

Thanks for this excellent explanatory post, Terry. Why is it, though, that number theoretic algorithms complexity is always done in terms of the logarithm of the number sizes? That is, the complexity of AKS on input n is polylogarithmic in n.

 **anoymous** · 12 August 2009

I guess because you want to have a running time polynomial in the input size, i.e., the number of bits you need to write down the input. For a single number it is $O(\log(n))$

 **Leandro** · 12 August 2009

Just a minor typo: right after equation (1), it says:
"..pick a few values of and see..". I think you mean:
"..pick a few values of n and see..".

[Actually, I meant a here, but thanks for the correction. -T.]

 **David Speyer** · 12 August 2009

"Why is it, though, that number theoretic algorithms complexity is always done in terms of the logarithm of the number sizes?"

Anonymous's answer, that this is the size of the input, is good. One should also note that it is the efficiency of the "easy" operations: addition, multiplication, division with remainder, GCD and so forth. Since these are the building blocks of our algorithms, it makes sense to study those algorithms which are not much worse than the pieces they are built from.

>  **Anonymous** · 12 August 2009
>
> Dear Terry,
>
> probably, the main question that one needs to solve is the following:
>
> suppose that we have all the set of a's that violate (4) and thus gurantee that p is a composite and not a prime power. What do you think: can we somehow use this set to obtain a factor of p?

 **Kenny Easwaran** · 12 August 2009

Very nice explanation of the results! I started to get a bit confused about half way through with the big-O notations. For instance, you state Theorem 1 as:

> Theorem 1 (AKS theorem) Suppose that for all $1 \leq a, r \leq O(\log^{O(1)} n)$, (4) holds, and a is coprime to n. Then n is either a prime, or a power of a prime.

The first sentence looks like it can be true for all n while of course the second isn't. After all, there is some constant such that (4) holds for all a, r less than that constant times $\log n$, namely 0.

I suppose the right way to understand this is that the big-O expressions are all taken to introduce existential quantifiers over constants somewhere early on, with scope over the entire theorem, rather than just scope over the specific equation the big-O statement is in. And you're also supposed to understand the scope of this quantifier with respect to various other quantifiers appropriately too. Thus, the more formal statement of Theorem 1 must say

> $\exists k \forall n$ If $\forall 1 \leq a, r \leq k(\log^k n)$ (4) holds, then n is either a prime, or a power of a prime.

Switching the initial existential and universal quantifiers yields a statement that is trivially true (just choose $k = 0$ if n is prime or a power of a prime and $k = n$ otherwise).

Perhaps the fact that this existential quantifier gets widest scope is implicit in the fact that it's called a constant, but it gets more unclear when you cross sentence boundaries. Because there are other examples with the same format as the statement of Theorem 1 here where the quantifier clearly isn't meant to have wide scope over both sentences. For instance

> Suppose one can factor n in $n^{O(1)}$ time. Then standard cryptography is weak.

This doesn't mean that there is some exponent on the polynomial that would mean cryptography is weak – rather it means that *any* exponent on the polynomial would work.

 **Jonathan Vos Post** · 14 August 2009

Thank you for the clarity in showing that PRIME is in P. Now, is SEMIPRIME in P? By definition, a semiprime is a product of exactly two primes, not necessarily distinct. They matter because of an important class of cryptosystems, in a multibillion dollar industry. The complication: there exist specific semiprimes in the literature that have been proven to be semiprimes, without any prime factorization know. We don't know the complexity class for prime factorization. So what is the complexity class for determining if a given integer is or is not a semiprime? Several experts in number theory and quantum computing have told me that mine is an interesting question, but probably very difficult. I mention quantum computing because SEMIPRIME might be in a qc complexity class of interest. What do you think?

Four Derandomization Problems « Combinatorics and more · 6 December 2009

[...] about primality certificate. and the AKS deterministic algorithm for primality is described in this post by Terry Tao (among other places). It was a beautiful combination of number theoretic ideas and derandomization [...]

Manindra Agrawal and Carlos Gustavo (Gugu) Moreira share 2010 TWAS prize in Mathematics « Disquisitiones Mathematicae · 3 November 2010
[...] algorithm to decide primality of natural numbers (for a nice exposition on this subject see this post by Terence Tao), and the works of Gugu (Annales de l'IHP, 1996), and Gugu and J.-C. Yoccoz (Annals of Math. [...]

 **Ricardo Menares** · 22 February 2011
Small typo: in the paragraph above (4) it should be (Z/NZ[x])/(X^r-1) instead of F_p[x]/(X^r-1)

[Corrected, thanks – T.]

 **amateur** · 27 September 2011
Hello I think that binomial test is also suitable for factoring.

binomial test – phrase used from pdf in chapter 11

Click to access prime.pdf

Just see next links and you will understand:

1. key : -sieve of eratosthenes connected to triangles line N, and thus giving factors for that line N, for either composite number (or prime)

2-key :–sieve of eratosthenes connected to remainders of binomial coefficients in pascal triangle by modulo n. (compare triangle from video and pascal triangle in mod N)

Image unavailable · Original image

Finally you will see that every midterm binomial coeficients, which are not divisible by N, correlate to factor of number N by its position and its value.

So bolded statement in next jpg is wrong

Image unavailable · Original image

 **amateur** · 30 September 2011
There is one more think that is of interest (for me). Aditional way of checking factors of explicit number N (for example 341)is that you calculate mod 341 while building pascal triangle to line 341.

You will se that numbers 11 and 31 have midterm values that are divisble by 341 so mod 341 is 0.

Image unavailable · Original image

Line 341 also gives interesting results

Image unavailable · Original image

The only problem finding factors of 341 this way is that mod 341 doesnt take its full function for smaller numbers (lines) and these factors my be "fake" for 341 (see again first picture: fake factor 2,3,5,7)

amateur · 3 October 2011

"You will se that numbers 11 and 31 have midterm values that are divisible by 341 so mod 341 is 0."

Wrongly put

"You will se that lines 11 and 31 have midterm values that are divisible by 11 or 31 in pascal triangle mod 341.

gabriel b · 31 October 2011

this is so complex that i can't understand anything. anyway i've made a simple program to find prime numbers by 'brute force dividing' that is pretty fast and completely foolproof, it would be good to use a program based on this AKS primality test that i see so much fuss about on the internet to compare the speed difference between this approach and my simple one.
i saw some project on sourceforge about this where you could download

the program and i did but i can't see any working executable there, nor any indications, so maybe it isn't "the real thing", anyway if someone knows about one executable implementation of this AKS method of discovering primes please respond to this question saying something because i'm going to be notified by e-mail.

Number Theory: Lecture 22 « Theorem of the week · 25 November 2011

[...] (The Higher Arithmetic) both have material on pseudoprimes and primality testing. Terry Tao has a blog post about the AKS primality test, with various links to further [...]

amateur bojan vasiljevic · 25 November 2011

How to find factors of 2 almost primes?
N=a*b

We know that digit sum of N = digit sum of a * digit sum of b
digit sum arithmetic: http://www.applet-magic.com/Digitsum.htm

example: digit sum 341 = digit sum (3+4+1) = 8
= digit sum (31) * digit sum (11) =4*2=8

As it shown in the next picture if there exists such N that there is N=a*b (not including that a or b might be 1 and N) than a remainder of mod N with the respect to its position equals N

see picture and last line of pascal triangle in mod 341 where cell of value 31 is showed in 11s position. 31*11=431

Image unavailable · Original image

so the algorithm for finding factors is very similiar to aks

We have mod N. Diferrence here is that aditional mod (in aks is mod (x to the power of r) -1) is mod 9 (digit sum equivalence). So binomial quoeficent in mod N and then (mod N * position) mod 9

And at last we check those cells with the respect of their position.

Example
For 341 we check all the cells where (value*position) mod 9=8 (because digit sum(341)=8)

amateur bv · 25 November 2011

Here is the case picture for number 341 where pascal triangle is in mod 341, so we look at line 341 all cells where (cell value * cell position) mod 9 = 8.

Image unavailable · Original image

bv · 25 November 2011

Now im interested in digt sum for binomial coefficients…..

amateur bv · 29 February 2012

An Algorithm For Factoring Integers that uses method very similar to AKS.

They say asymptotic computational complexity of their method is unknown!?…..

http://www.google.si/url?sa=t&rct=j&q=&esrc=s&source=web&cd=1&ved=0CBsQFjAA&url=http%3A%2F%2Feprint.iacr.org%2F2012%2F097.pdf&ei=laFOT77KLIresgbXh7C7Dw&usg=AFQjCNGIlesJ5E6NVf3Bu2wyOYErxB-lAA

An Algorithm For Factoring Integers
Yingpu Deng and Yanbin Pan

Number Theory: Lecture 22 | Theorem of the week · 29 November 2013
[…] (The Higher Arithmetic) both have material on pseudoprimes and primality testing. Terry Tao has a blog post about the AKS primality test, with various links to further reading. Andrew Granville wrote an […]

Blog 2 – Algorithms – Site Title · 19 September 2016
[…] Tao, Terence. “The AKS Primality Test.” Whtats New. WordPress, 23 Feb. 2011. Web. 19 Sept. 2016. https://terrytao.wordpress.com/2009/08/11/the-aks-primality-test/ […]

Blog 6-Mathematics and Computer Science – Site Title · 14 November 2016
[…] Tao, Terence. “The AKS Primality Test.” Whtats New. WordPress, 23 Feb. 2011. Web. 19 Sept. 2016. https://terrytao.wordpress.com/2009/08/11/the-aks-primality-test/ […]

anonowski · 7 April 2020

“[…] where {\phi_m: F \rightarrow F} is the ring homomorphism that sends {X} to {X^m}.”
How do we know that this gives a well defined ring homomorphism?

> **Terence Tao** · 7 April 2020
>
> Ah, one needs to specify that m is coprime to r, so that X^m is also a primitive r^{th} root of unity. I’ve added this to the post.

Anonymous · 20 April 2020

Can we tell primality of (x+1)(x+2)-x when x is an integer?

Gautam Tripathi · 12 February 2022
Regarding this comment…

“But if {n} is divisible by some smaller prime {p}, then by setting {i} equal to the largest power of {p} that divides {n}, one sees that {\binom{n}{i}} is not divisible by enough powers of {p} to be divisible by {n}, a contradiction.”

Isn’t setting {i} to any prime factor of {n} sufficient to show that {\binom{n}{i}} is not divisible by {n}?



Anonymous · 20 November 2023

I think so too, since $\binom{n}{p}$ is nonzero modulo n: if you set i equal to p, then the binomial fraction $\binom{n}{p}$ is divisible by at most p^{j-1}, where j is the largest exponent of p in the factorisation of n